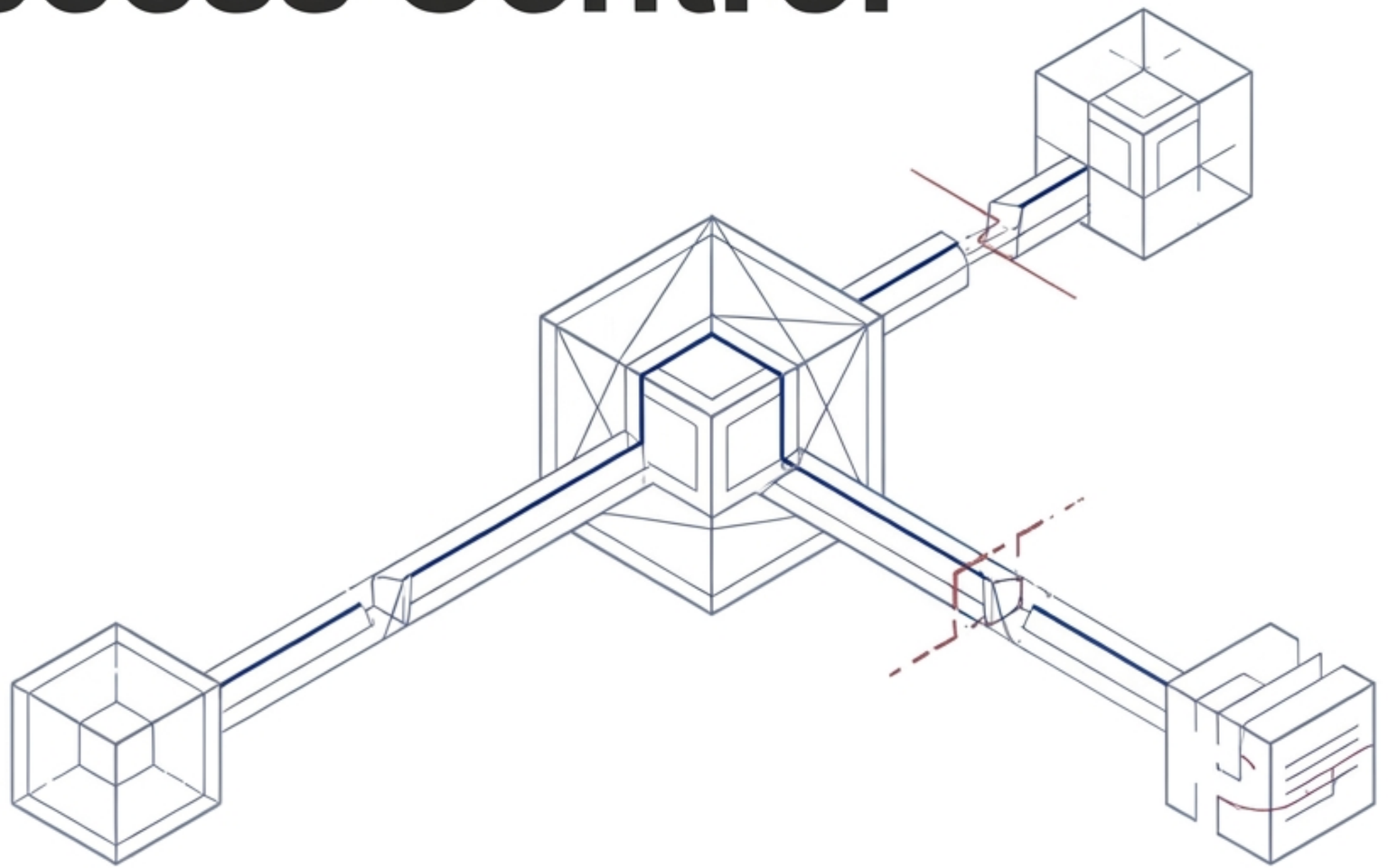
OWASP Top 10 A01:2021

Broken Access Control

Brandon Fuentes Micete



El riesgo número uno en la web.

El control de acceso roto (BAC) no es un error de código, es un fallo arquitectónico. Las métricas demuestran que priorizamos verificar quién es el usuario, pero sistemáticamente olvidamos validar qué puede hacer.

94% de las aplicaciones evaluadas presentan este riesgo (318,487 ocurrencias documentadas, tasa de incidencia 3.81%).

Posición #5 (2017)

#1 (2021)

La ilusión de la sesión segura

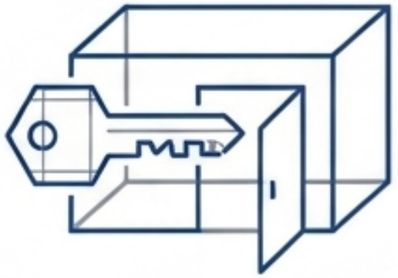


Autenticación (Quién eres)

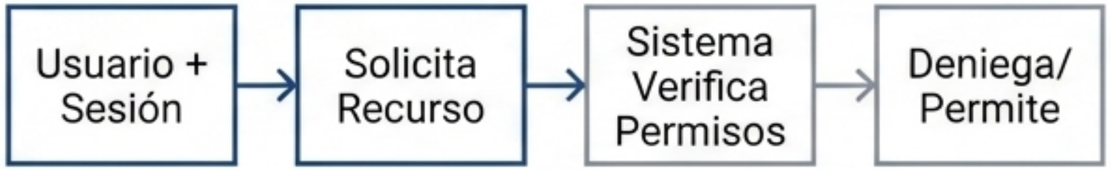


El usuario ha iniciado sesión correctamente.

(Falla cubierta en A07).



Autorización (Qué puedes hacer)



El usuario logueado, ¿es el propietario legítimo de este recurso específico?

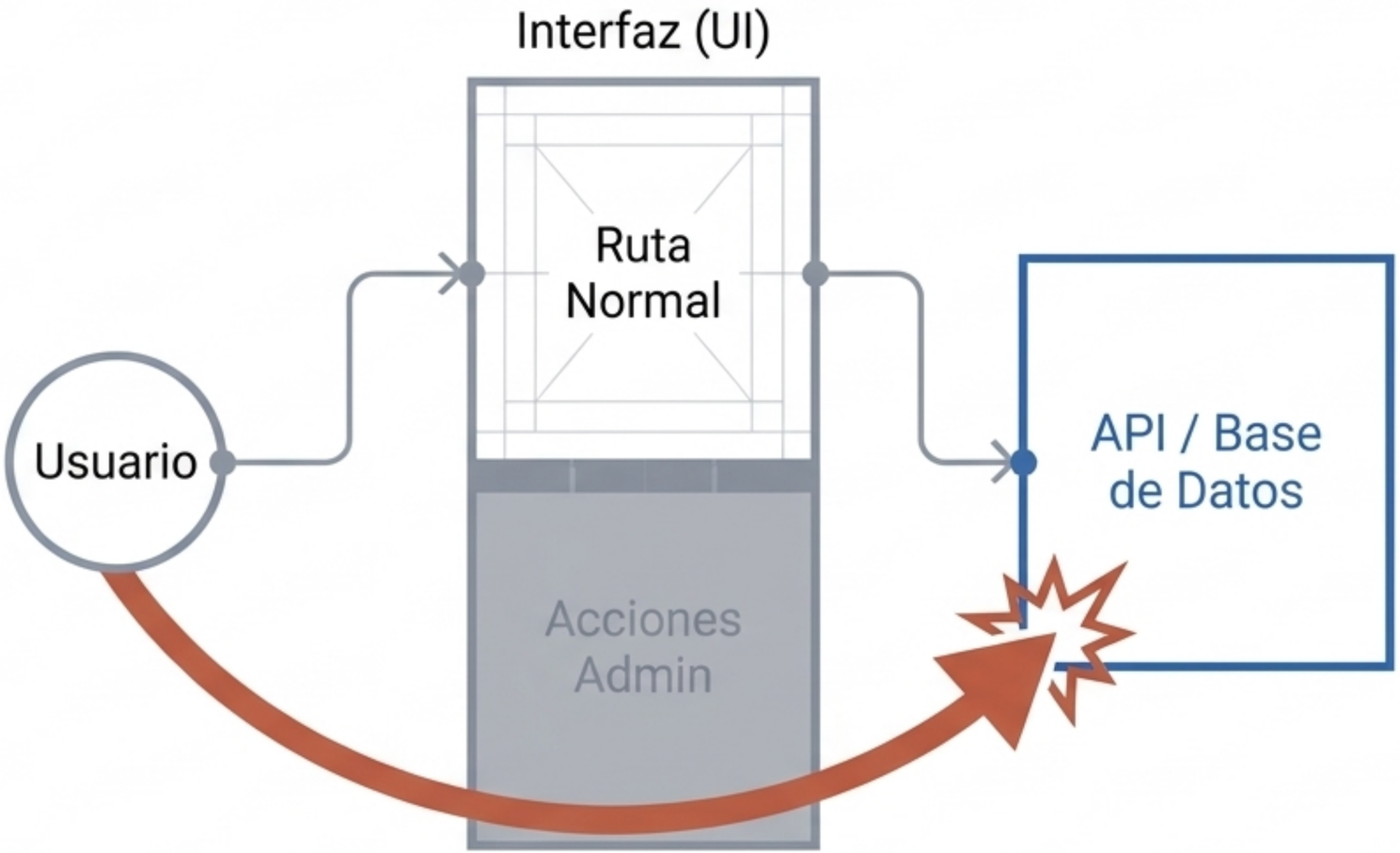
(La raíz de A01).

La anatomía de un bypass (Navegación Forzada)

Los desarrolladores asumen que el usuario interactuará solo con lo que la interfaz expone.

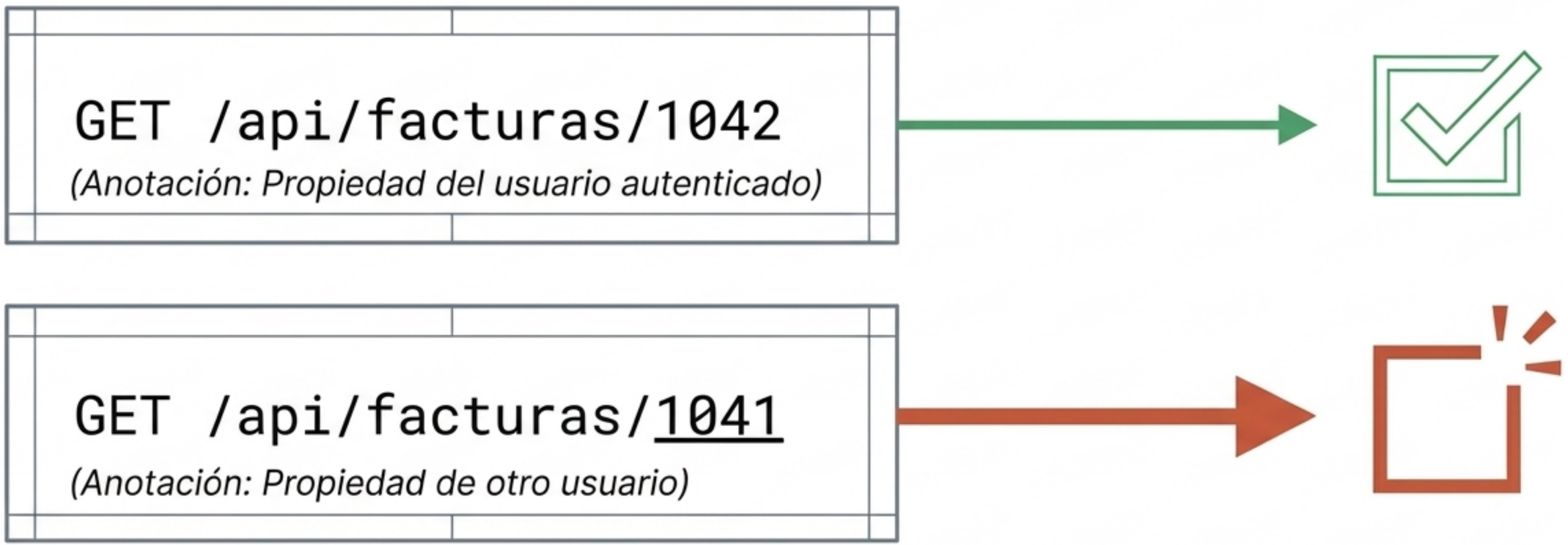
El atacante ignora la UI y manipula directamente los identificadores, rutas o tokens en la petición HTTP.

La validación del lado del cliente no es un control de seguridad.



Escenario de Ataque: Referencia Directa a Objetos (IDOR)

El sistema confía ciegamente en el parámetro de la URL (el ID 1041) y lo inyecta directamente en la consulta de base de datos. Falla al no validar que el identificador del objeto solicitado pertenezca a la sesión actual.



El principio de defensa: Denegación por defecto

Confianza Cero en el Cliente

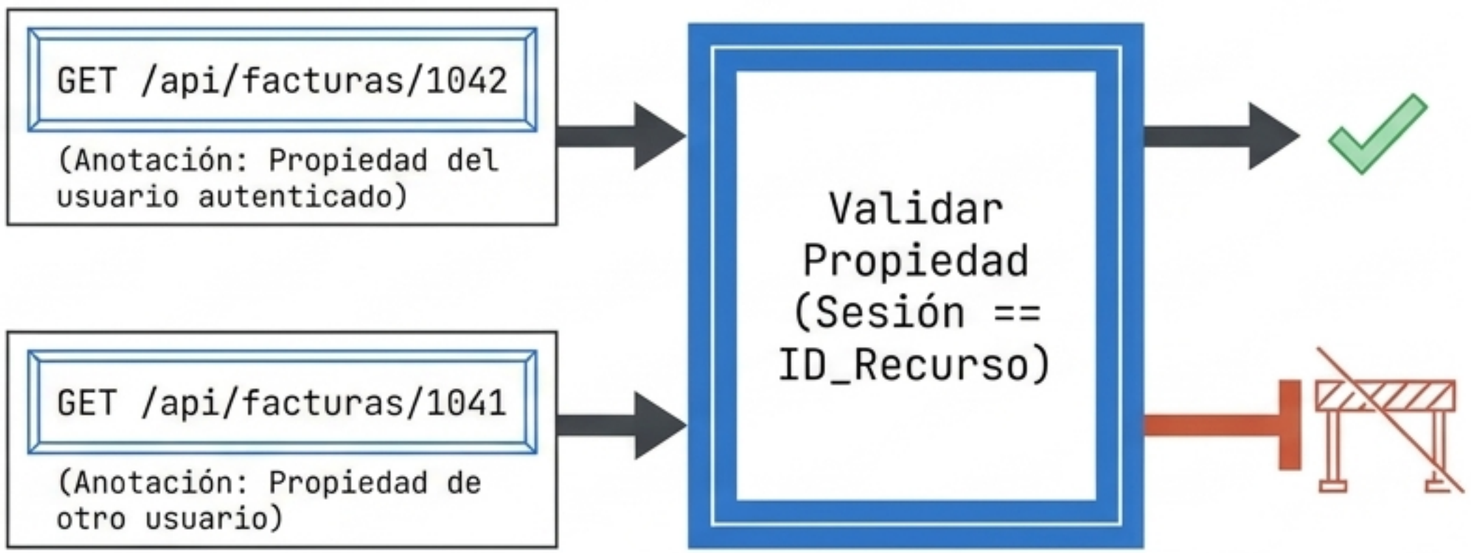
Toda verificación de roles y propiedad debe ocurrir obligatoriamente en el backend.

Centralización

Utilizar un framework único de autorización, no comprobaciones de roles dispersas en los controladores.

Autorización a Nivel de Objeto

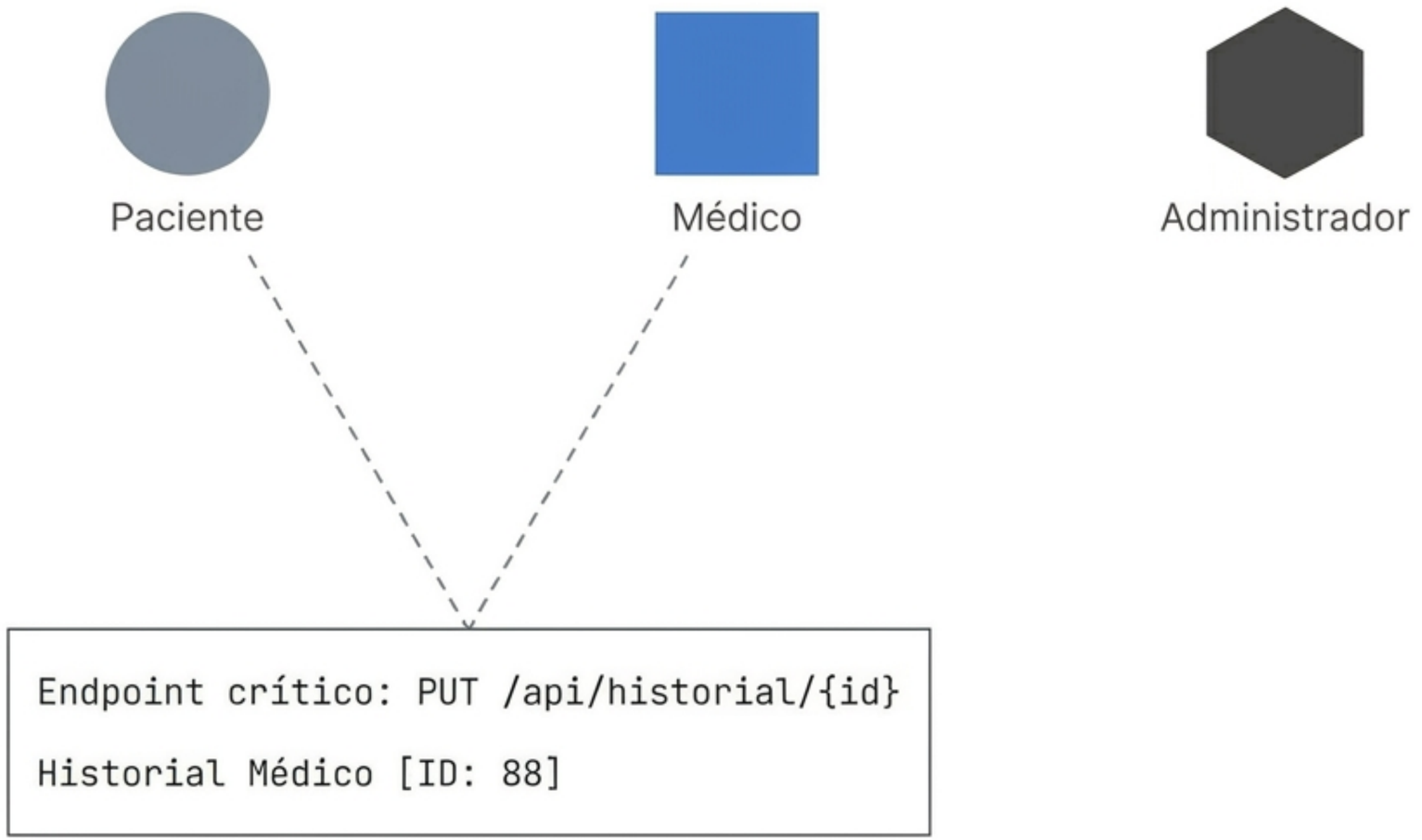
No basta con saber si el rol es Usuario; debe verificarse si es El dueño exclusivo de la Factura X.



Reto Arquitectónico .NET (Análisis Conceptual)

Sistema Hospitalario.
Tres roles aislados: Paciente, Médico, Administrador.

La Regla de Negocio:
Un médico solo puede editar el historial de los pacientes asignados especificamente a él. Un paciente solo puede leer su propio historial.



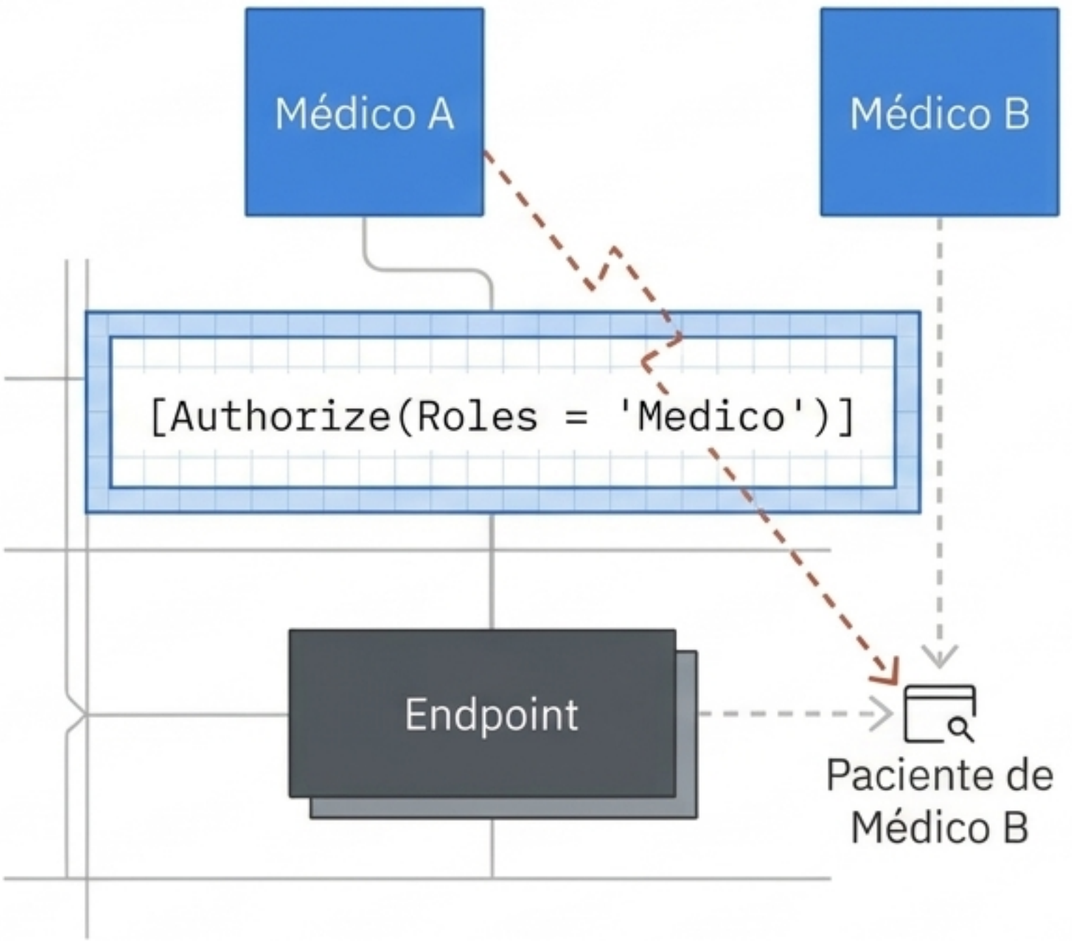
Análisis de Políticas: ¿Dónde está el error?

Implementación Actual

Protegemos el endpoint con una política genérica basada en roles: Solo los usuarios con rol Médico pueden editar.

El Fallo (Escalamiento Horizontal)

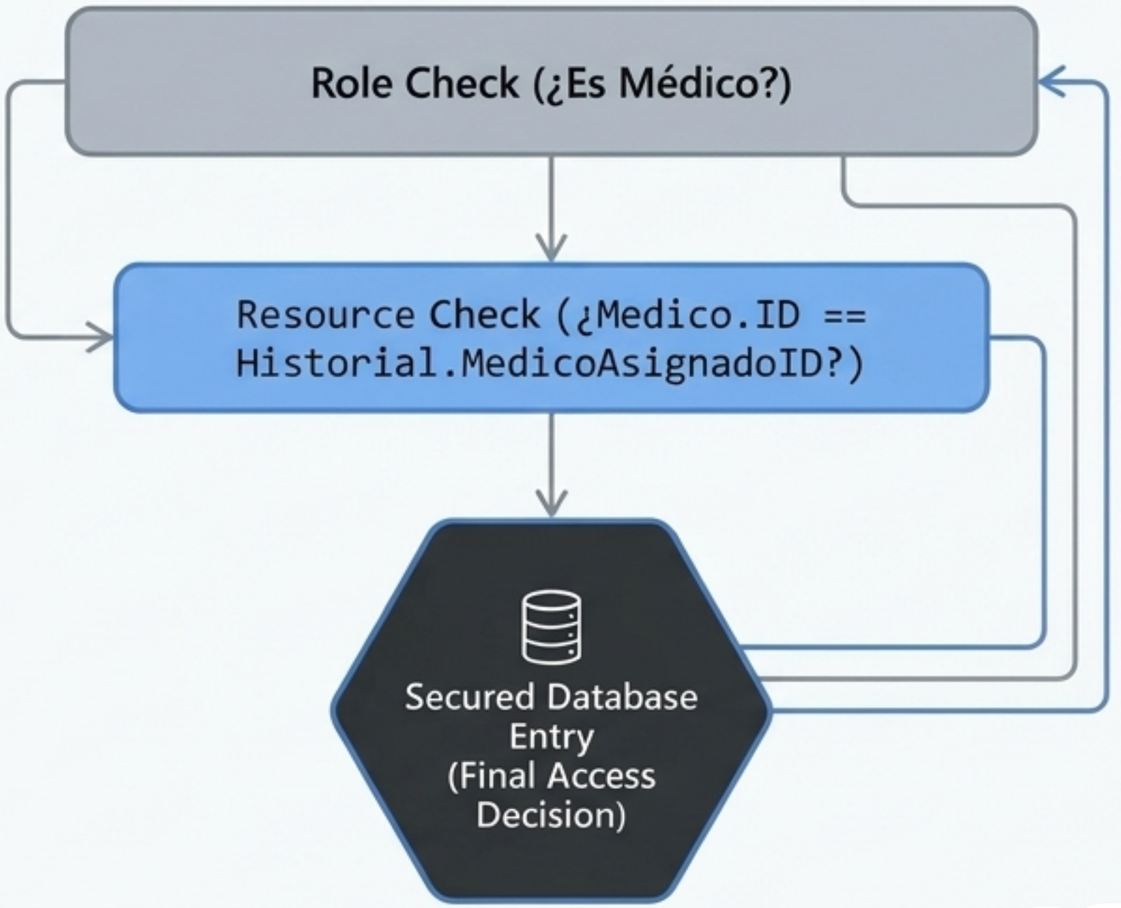
El sistema valida el rol (Efectivamente, el usuario es un Médico), pero omite el contexto del dato. El Médico A acaba de alterar exitosamente el expediente del paciente del Médico B.



Resolución: Autorización Basada en Recursos (IAuthorizationService)

El Nuevo Diseño

1. **Desacoplar:** Separar la validación del rol de la validación del recurso.
2. **Diseñar un Requisito (Requirement):** En .NET, inyectar el ID del recurso y el contexto de seguridad del usuario antes de ejecutar la acción.
3. **Evaluación Integral**: El sistema ahora exige (Autenticado) + (Es Médico) + (Es SU paciente asignado).



El paradigma del acceso seguro

Denegar por Defecto

Si el acceso a un recurso no está explícitamente permitido por una política centralizada, está terminantemente prohibido.

El Cliente Miente

Nunca confíes en parámetros, IDs en URLs, o tokens que no estén verificados y firmados criptográficamente por el servidor.

Contexto sobre Rol

La autorización moderna trasciende el RBAC. Exige validar la relación exacta entre la identidad del usuario y el registro especifico en la base de datos.